

RAG / Organization-Specific AI Implementation

Baseline Architecture & Integration Guide for Angular + Django REST + PostgreSQL + AI/RAG

Target Audience: AI Coding Agents & System Architects | Version: 1.0 Baseline | Status: Approved Implementation Architecture

DOCUMENT PURPOSE

This document serves as an implementation handoff for an AI coding and development agent. Its purpose is to guide the rebuilding of the TNA AI architecture so that retrieval, generation, assessment, competency analysis, and organizational reporting are strictly grounded in the organization's actual database and approved documents. The design prioritizes accuracy, traceability, authorization, source grounding, and testability.

SOURCE-BASED BACKEND OBSERVATIONS

The supplied backend views show an existing Django REST structure with `User`, `Role`, `Department`, `Position`, `TrainingProvider`, `TrainingProgram`, `TrainingEnrollment`, and `TrainingEvaluation` concepts. The accounts API already applies role-aware visibility: HR/Admin can see all users, Department Heads can see themselves and their subordinates, and ordinary employees can see only themselves.

The training API currently exposes `TrainingProvider`, `TrainingProgram`, `TrainingEnrollment`, and `TrainingEvaluation`. TrainingProgram creation is restricted to authenticated admin users in the supplied view, while enrollment visibility is broader for HR/Admin and limited to the employee for ordinary users.

Crucial Requirement: These existing permissions must be preserved and extended to the AI layer; the LLM must never become the authorization mechanism.

1. TARGET ARCHITECTURE

The target architectural stack follows the flow: **Angular** → **Django REST API** → **Domain Apps + AI Orchestration** → **PostgreSQL + pgvector** → **LLM**. The AI layer reads authorized structured facts from PostgreSQL and retrieves approved unstructured knowledge from pgvector, subsequently generating a response grounded in both sources.

- Key Principle:**
- **Database** = Authoritative structured facts
 - **RAG** = Authoritative searchable document knowledge
 - **Deterministic Django Services** = Calculations and permissions
 - **LLM** = Interpretation, synthesis, and generation

2. DJANGO APPS AND AI INTEGRATION

App	AI Relationship	Examples & Scope
accounts	Context + Authorization	Employee records, role definitions, department assignments, position titles, supervisor hierarchy.
organizations	Organization Context	Department structures, organizational hierarchy, position mappings.
competencies	Core Analytical Source	Required competency framework, current levels, gap analysis.
tna	Training-Need Context	Training requests, justifications, approval states, identified gaps.
training	Primary RAG Source	Programs, modules, sessions, trainer materials, enrollments.

App	AI Relationship	Examples & Scope
assessments	Learning Evidence	Question pools, user answers, attempts, deterministic scores, final assessments.
ai	Orchestration Layer	Retrieval logic, prompting, generation, output evaluation, audit trails.
reports	Presentation Layer	HOD/HR training analytics and organizational intelligence reports.
notifications	Delivery System	Notifying HODs, HR, and employees about report availability and learning events.

3. DO NOT DUPLICATE THE WHOLE DATABASE INTO VECTOR STORAGE

Do not embed every employee row, password, authentication record, permission record, or transactional database row. Keep those strictly in PostgreSQL. RAG should primarily index approved documents and training knowledge. When the AI needs live employee, department, or score facts, the AI service should query authorized Django services directly.

4. KNOWLEDGE INGESTION PIPELINE

The ingestion process follows a structured workflow:

1. Document Upload & Validation: Trainer/HR uploads PDF, DOCX, PPTX or approved organizational documents → Validate file integrity and user permissions.

2. Extraction & Chunking: Extract plain text → Preserve document metadata → Split text into semantic chunks.

3. Vectorization & Indexing: Generate embeddings → Store chunks and vectors in `pgvector` → Link each chunk to its source document, department, training program/module/session, version, approval status, and access scope.

Every chunk must retain provenance. **Minimum Required Metadata:** `document_id`, `chunk_id`, page/slide/section (where available), department scope, `training_program_id`, `training_module_id`, `training_session_id`, document version, approval status, `created_at`, and `embedding_model`.

5. RETRIEVAL PIPELINE

The retrieval workflow is executed in sequential stages:

User question → Authenticate user → Determine access scope → Classify intent → Build retrieval query → Retrieve candidate chunks → Apply metadata/permission filters → Rerank candidates → Select context → Generate answer → Validate grounding → Return answer + citations/source references.

Retrieval must be **hybrid** where practical: combining semantic vector similarity with keyword/metadata filtering. Do not rely on vector similarity alone for exact organizational terms, codes, competency names, policy identifiers, or technical terminology.

6. ACCESS CONTROL BEFORE RETRIEVAL

The system must calculate the user's authorized scope *before* searching knowledge. For example, an employee should not retrieve another employee's private assessment. A Department Head should be restricted to their department/subordinates where business rules require it. HR/Admin may have wider access.

Implement a permission-aware retrieval function such as `retrieve_knowledge(user, query, filters)`, where `filters` are generated by trusted Django authorization code—**never by the LLM**.

7. RAG CONTEXT ASSEMBLY

Context should contain only the smallest relevant evidence needed for the task. Include source labels and identifiers so the generation layer can cite the evidence. Avoid dumping large amounts of unrelated text into the prompt.

For training questions, prioritize current training program/module/session content and approved trainer material. For organization questions, prioritize current approved policies, procedures, competency frameworks, job descriptions, and strategic documents.

8. LLM ROLE & LIMITATIONS

The LLM should explain, summarize, generate grounded questions, interpret assessment evidence, compare competency results, and produce management narratives. It **must not** invent organizational facts, calculate authoritative scores, bypass permissions, approve training requests, or create official curriculum without human approval.

If evidence is insufficient, the model should explicitly respond that the available organizational sources do not contain enough information instead of guessing.

9. TRAINING-SPECIFIC RAG FLOW

Employee requests training → HOD approves → HR/authorized training manager creates `TrainingProgram` → Trainer is assigned → Trainer uploads official material → Material is indexed → Modules/sessions are mapped → Employee enrolls → Daily session opens → AI retrieves only relevant approved material → AI explains topics and generates grounded quiz → Results stored → Weak topics detected → Final assessment generated from full approved curriculum → Competency/impact analysis → HOD/HR report generated.

10. DAILY QUIZ GENERATION

Inputs: Employee context, training program, current session/day, module, learning objectives, approved source chunks, prior performance, desired difficulty, and question count.

Output: Questions, options (where applicable), correct answer/rubric, explanation, source chunk IDs, difficulty rating, competency/topic tag, and generation model/version.

Question generation must be source-grounded. A generated question should be rejected or regenerated if its answer cannot be supported by the retrieved approved material or explicitly supplied structured data.

11. ADAPTIVE LEARNING & REMEDIATION

After each quiz, calculate scores deterministically via Django backend services. Map incorrect answers to target topic/competency. The AI interprets weak areas and proposes targeted remediation. The system can retrieve specific supplemental content and generate additional practice questions. Always store the remediation decision and evidence.

12. FINAL ASSESSMENT

The final assessment must cover the approved program curriculum, not arbitrary model pre-training knowledge. It should contain technical, practical, scenario-based, and organizational workflow questions. Store source references and competency mappings for every generated question.

13. RECOMMENDED DATABASE ADDITIONS

Model Name	Purpose & Function
<code>KnowledgeDocument</code>	Uploaded approved document records, file paths, and high-level metadata.
<code>KnowledgeChunk</code>	Chunked source text with full provenance tracking and page/section references.
<code>KnowledgeEmbedding</code>	Vector representation linked to chunks (may be integrated into <code>KnowledgeChunk</code>).

Model Name	Purpose & Function
AIRequest	Complete audit trail logging every AI operation, input payload, and caller context.
AIResponse	Persists LLM output, execution status, latency metrics, and error metadata.
AIQuizGeneration	Stores generated quiz sets, configuration rules, and context inputs.
AIQuestionEvidence	Explicitly links each generated question to its supporting source chunks.
AICompetencyAnalysis	Stores AI interpretation of competency evidence and identified skill gaps.
AITrainingImpactReport	Generates training effectiveness, improvement metrics, and organizational relevance.
AIAuditLog	Security audit trail for tracking permission checks and data access attempts.
AIModelConfig	Configuration table managing AI providers, model IDs, and parameter defaults.
PromptTemplate	Versioned system prompts, instructions, and output schema specifications.

14. MODEL SELECTION & EMBEDDINGS

Start with a capable instruct model rather than training from scratch. A Qwen-class instruct model is a reasonable candidate for an initial self-hosted architecture; the exact model size must be benchmarked against your available hardware, context length, latency, and concurrent-user requirements.

Use a separate dedicated embedding model for vector retrieval. Evaluate embedding quality on your organization's specific terminology before production. Do not assume the LLM's embedding capabilities are automatically optimal for document chunk retrieval.

15. FINE-TUNING STRATEGY

Do not fine-tune on raw organizational documents as the first implementation step. Use RAG for dynamic and changing knowledge. Fine-tuning can later specialize behavior using approved historical examples (e.g., high-quality training analyses, specific question formats, report styles, and classification tasks). Consider LoRA/QLoRA/PEFT techniques only after empirical evaluation proves it is necessary.

16. ACCURACY & EVALUATION REQUIREMENTS

The agent implementing this system must create an evaluation set before declaring the RAG system accurate. Build representative test questions across: policies, procedures, positions, competencies, training materials, daily training questions, employee performance analysis, and HOD organizational queries.

For each test question record: expected evidence/source, acceptable answer characteristics, target access scope, and whether the answer must be deterministic or generative.

Minimum Tracking Metrics: Retrieval recall/precision on labeled test cases, source attribution correctness, grounded-answer rate, unsupported-claim rate, question-answer validity, permission leakage rate, and end-to-end response latency.

17. HALLUCINATION DEFENSE MECHANISMS

Implement defense layers: mandatory source context, minimum retrieval confidence thresholds, strict metadata filters, cross-encoder reranking, structured output JSON schemas, source citations, explicit refusal/insufficient-evidence behavior, and post-generation validation checks. For high-risk management reports, require human review before publication.

18. OBSERVABILITY & AUDITING

Log request ID, user ID, operation type, model/version, prompt version, retrieved source IDs, retrieval scores, generation status, latency, token usage where available, validation result, and approval status. **Do not log passwords or unnecessary sensitive PII data.**

19. AGENT IMPLEMENTATION ORDER

1. Inspect existing Django models, serializers, URLs, permissions, and migrations before changing architecture.
2. Define AI domain models and create initial database migrations.
3. Implement secure document storage and metadata parsing.
4. Implement text extraction and semantic chunking modules.
5. Implement embedding generation and `pgvector` integration.
6. Implement permission-aware retrieval filters.
7. Implement unified LLM gateway service.
8. Implement grounded Q&A with strict citation verification.
9. Implement training-session-specific retrieval pipelines.
10. Implement daily quiz generation with evidence chunk links.
11. Implement assessment analysis and deterministic score integrations.
12. Implement competency gap and organizational impact reporting.
13. Build automated RAG evaluation suite and benchmarking scripts.
14. Apply comprehensive security testing, monitoring, and audit verification.

20. AGENT ACCEPTANCE CRITERIA

The implementation is acceptable only when:

- Every retrieved source is strictly permission-filtered based on user auth.
- Training answers are demonstrably grounded in approved trainer material.
- Generated questions have traceable, verified evidence links.
- Authoritative numerical calculations come directly from Django/database logic.
- Unsupported questions reliably produce an insufficient-evidence response.
- HOD/HR reports clearly display supporting evidence behind important claims.
- All AI actions and context retrievals are logged in audit trails.
- No cross-user or cross-department data leakage occurs under any context.
- The system successfully passes a repeatable automated evaluation suite.

21. FIRST TEST SCENARIO FOR TOMORROW

Use one controlled training program (e.g., *Advanced Excel*). Upload one approved trainer document. Create 5–10 representative questions whose answers are explicitly present in the document. Verify extraction, chunking, retrieval, source attribution, answer grounding, and proper refusal when the answer is absent. Then create Day 1 material and test daily quiz generation. Only after this passes should the agent scale to multiple documents and organization-wide data.

22. RECOMMENDED END STATE & HANDOFF NOTE

Target End State Architecture:

Organization AI = Authorized PostgreSQL facts + Permission-aware RAG knowledge + Deterministic TNA analytics + Capable instruct LLM + Evidence validation + Audit trail.

This architecture allows the AI to understand the organization's structure and training knowledge without pretending that model weights are the database. Knowledge can be updated by uploading/approving new documents, while employee facts, competency scores, enrollments, and permissions remain strictly controlled by Django/PostgreSQL.

Final Handoff Note: The implementing agent should treat this document as the RAG architecture baseline, but must inspect the complete project models, serializers, URLs, settings, migrations, and database schema before writing code. The supplied views confirm only part of the current backend; missing model definitions must not be guessed.